



**UNIVERSIDAD PRIVADA DE TACNA**  
**FACULTAD DE INGENIERÍA**  
**Escuela Profesional de Ingeniería de Sistemas**

**PROYECTO DE UNIDAD N° 02**

**“Menor error en Validación Cruzada”**

Curso: MACHINE LEARNING

Docente: Ing. ISRAEL NAZARETH CHAPARRO CRUZ

**Diego Fernando Castillo Mamani (2022073895)**

**Cesar Nikolas Camac Melendez (2022074262)**

**Sergio Alberto Colque Ponce (2022073503)**

**Tacna – Perú**  
**2026**



## ÍNDICE

|                                                        |   |
|--------------------------------------------------------|---|
| I. INFORMACIÓN GENERAL                                 | 4 |
| - Objetivos                                            | 4 |
| - Equipos, materiales, programas y recursos utilizados | 4 |
| II. MARCO TEORICO                                      | 4 |
| III. PROCEDIMIENTO                                     | 5 |
| IV. ANALISIS E INTERPRETACION DE RESULTADOS            | 7 |
| V. CUESTIONARIO                                        | 8 |
| CONCLUSIONES                                           | 8 |
| RECOMENDACIONES                                        | 8 |
| BIBLIOGRAFÍA                                           | 8 |
| WEBGRAFIA                                              | 8 |



## INFORME DE LABORATORIO N° 02

### TEMA: Menor error en Validación Cruzada

#### I. INFORMACIÓN GENERAL

##### **Objetivo General:**

Realizar un análisis exploratorio de datos, preparar los datos para algoritmos de Machine Learning, generar pipelines de procesamiento, entrenar modelos de regresión y comparar su desempeño mediante el Error Absoluto Medio (MAE) y Validación Cruzada sobre el dataset California Housing.

##### **Objetivos Específicos:**

- Aplicar técnicas de análisis exploratorio para comprender la distribución y calidad de los datos del dataset California Housing.
- Implementar un pipeline de preprocesamiento que incluya imputación de valores nulos, escalado de características y codificación de variables categóricas.
- Crear nuevas variables derivadas (feature engineering) que mejoren la capacidad predictiva de los modelos.
- Entrenar y comparar cuatro modelos de Machine Learning: Regresión Lineal, Árbol de Decisión, Random Forest y Gradient Boosting.
- Optimizar el modelo de mejor desempeño mediante búsqueda aleatoria de hiperparámetros (RandomizedSearchCV).
- Evaluar el modelo final en un conjunto de prueba independiente para obtener una estimación imparcial del error.



- **Equipos, materiales, programas y recursos utilizados:**

| Recurso    | Detalle                              | Versión     |
|------------|--------------------------------------|-------------|
| Plataforma | Google Colaboratory (Colab)          | Web         |
| Lenguaje   | Python                               | 3.10+       |
| Librería   | pandas                               | 2.x         |
| Librería   | numpy                                | 1.x         |
| Librería   | scikit-learn                         | 1.x         |
| Librería   | matplotlib                           | 3.x         |
| Dataset    | California Housing (GitHub - ageron) | housing.tgz |
| Hardware   | CPU Google Colab (gratuito)          | -           |

## II. MARCO TEÓRICO

- **Machine Learning Supervisado - Regresión**

El aprendizaje automatico supervisado para regresion consiste en entrenar un modelo a partir de ejemplos etiquetados, donde cada instancia contiene un conjunto de variables de entrada (features) y una variable de salida numerica continua (target). El objetivo es aprender una funcion  $f(X)$  que minimice el error de prediccion sobre datos no vistos. En este proyecto, la variable objetivo es median\_house\_value, el precio mediano de las viviendas por bloque censal en California.

- **Error Absoluto Medio (MAE)**

El MAE (Mean Absolute Error) es una métrica de evaluación para modelos de regresión que calcula el promedio de las diferencias absolutas entre los valores predichos y los valores reales. Su fórmula es:

$$\text{MAE} = (1/n) * \text{sum}(|y_i - y_{\text{pred}_i}|)$$

Se eligió el MAE como métrica principal porque es robusto frente a errores grandes (a diferencia del RMSE), es interpretable directamente en dólares americanos (la misma unidad que la variable objetivo), y permite comparar modelos de forma intuitiva.

- **Validación Cruzada (Cross-Validation)**

La validación cruzada es una técnica de evaluación que divide el conjunto de entrenamiento en k subconjuntos (folds). El modelo se entrena k veces, cada vez usando k-1 folds para entrenamiento y 1 fold para validación. El error final es el promedio de los k errores obtenidos. En este proyecto se uso k=5 (5-fold cross-validation). Esta técnica es superior a una simple división train/test porque:

- Reduce la varianza de la estimacion del error.
- Aprovecha todos los datos disponibles tanto para entrenamiento como para evaluacion.
- Detecta el sobreajuste (overfitting) al evaluar en datos no vistos en cada iteracion.

- **Modelos Utilizados**

- **Regresión Lineal**

Modelo paramétrico que asume una relación lineal entre las variables de entrada y la variable objetivo. Sirve como línea base (baseline) para comparar modelos más complejos. Su ventaja es la interpretabilidad; su desventaja es la incapacidad para capturar relaciones no lineales.

- **Arbol de Decision**

Modelo no paramétrico que divide el espacio de características en regiones rectangulares mediante reglas de decisión. Puede capturar relaciones no lineales e interacciones entre variables. Tiende al sobreajuste si no se controla su profundidad máxima.

- **Random Forest**

Modelo no paramétrico que divide el espacio de características en regiones rectangulares mediante reglas de decisión. Puede capturar relaciones no lineales e interacciones entre variables. Tiende al sobreajuste si no se controla su profundidad máxima.

- **Gradient Boosting**

Método de ensamblaje secuencial que construye arboles de forma iterativa, donde cada nuevo árbol corrige los errores del modelo anterior. Es poderoso pero más susceptible al sobreajuste si no se ajustan correctamente sus hiperparametros.

- **Ingeniería de Características (Feature Engineering)**

La ingeniería de características consiste en crear nuevas variables a partir de las existentes para mejorar la capacidad predictiva del modelo. Las variables originales (`total_rooms`, `total_bedrooms`, `population`) son valores absolutos por bloque censal, que no son comparables entre bloques de diferente tamaño. Se crearon ratios relativos que son más informativos:

- `rooms_per_household`: promedio de habitaciones por hogar en el bloque. Indica la amplitud típica de las viviendas.
- `bedrooms_ratio`: proporción de dormitorios sobre el total de habitaciones. Valores bajos indican casas más espaciales.
- `people_per_household`: promedio de personas por hogar. Relacionado con la densidad familiar.

- **Pipeline de Preprocesamiento**

Un pipeline de scikit-learn encadena transformaciones y el modelo en un flujo reproducible. Garantiza que las transformaciones de entrenamiento se apliquen correctamente al conjunto de prueba sin filtrar información del futuro (data leakage). El pipeline implementado incluye:

- `SimpleImputer` con estrategia 'median': reemplaza valores nulos de `total_bedrooms` (207 registros) con la mediana, siendo robusta frente a outliers.
- `StandardScaler`: estandariza las variables numéricas a media 0 y desviación estándar 1, necesario para modelos sensibles a la escala.
- `OneHotEncoder`: convierte la variable categórica `ocean_proximity` en columnas binarias (0/1).

### III. PROCEDIMIENTO

#### 1. Carga de Datos

Se implementó una función `load_housing_data()` que descarga automáticamente el dataset California Housing desde el repositorio público de GitHub (ageron/handson-ml) si no existe localmente. El dataset contiene 20,640 registros y 10 variables: 9 numéricas y 1 categórica (`ocean_proximity`). Esta automatización garantiza la reproducibilidad del experimento en cualquier entorno.

```
def load_housing_data():  
    tarball_path = Path('datasets/housing.tgz')  
    if not tarball_path.is_file():  
        url = 'https://github.com/ageron/data/raw/main/housing.tgz'  
        urllib.request.urlretrieve(url, tarball_path)  
    with tarfile.open(tarball_path) as housing_tarball:  
        housing_tarball.extractall(path='datasets')  
    return pd.read_csv(Path('datasets/housing/housing.csv'))
```

```
def load_housing_data():  
    tarball_path = Path('datasets/housing.tgz')  
    if not tarball_path.is_file():  
        Path('datasets').mkdir(parents=True, exist_ok=True)  
        url = 'https://github.com/ageron/data/raw/main/housing.tgz'  
        urllib.request.urlretrieve(url, tarball_path)  
    with tarfile.open(tarball_path) as housing_tarball:  
        housing_tarball.extractall(path='datasets')  
    return pd.read_csv(Path('datasets/housing/housing.csv'))  
  
housing = load_housing_data()  
housing.head()
```

... /tmp/ipykernel\_3761/4079493042.py:8: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject files or modify their metadata. Use the filter argument to control this behavior.  
housing\_tarball.extractall(path='datasets')

|   | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | population | households | median_income | median_house_value | ocean_proximity |
|---|-----------|----------|--------------------|-------------|----------------|------------|------------|---------------|--------------------|-----------------|
| 0 | -122.23   | 37.88    | 41.0               | 880.0       | 129.0          | 322.0      | 126.0      | 8.3252        | 452600.0           | NEAR BAY        |
| 1 | -122.22   | 37.86    | 21.0               | 7099.0      | 1106.0         | 2401.0     | 1138.0     | 8.3014        | 368500.0           | NEAR BAY        |
| 2 | -122.24   | 37.85    | 52.0               | 1467.0      | 190.0          | 496.0      | 177.0      | 7.2574        | 352100.0           | NEAR BAY        |
| 3 | -122.25   | 37.85    | 52.0               | 1274.0      | 235.0          | 558.0      | 219.0      | 5.6431        | 341300.0           | NEAR BAY        |
| 4 | -122.25   | 37.85    | 52.0               | 1627.0      | 280.0          | 565.0      | 259.0      | 3.8462        | 342200.0           | NEAR BAY        |



## 2. Análisis Exploratorio de Datos (EDA)

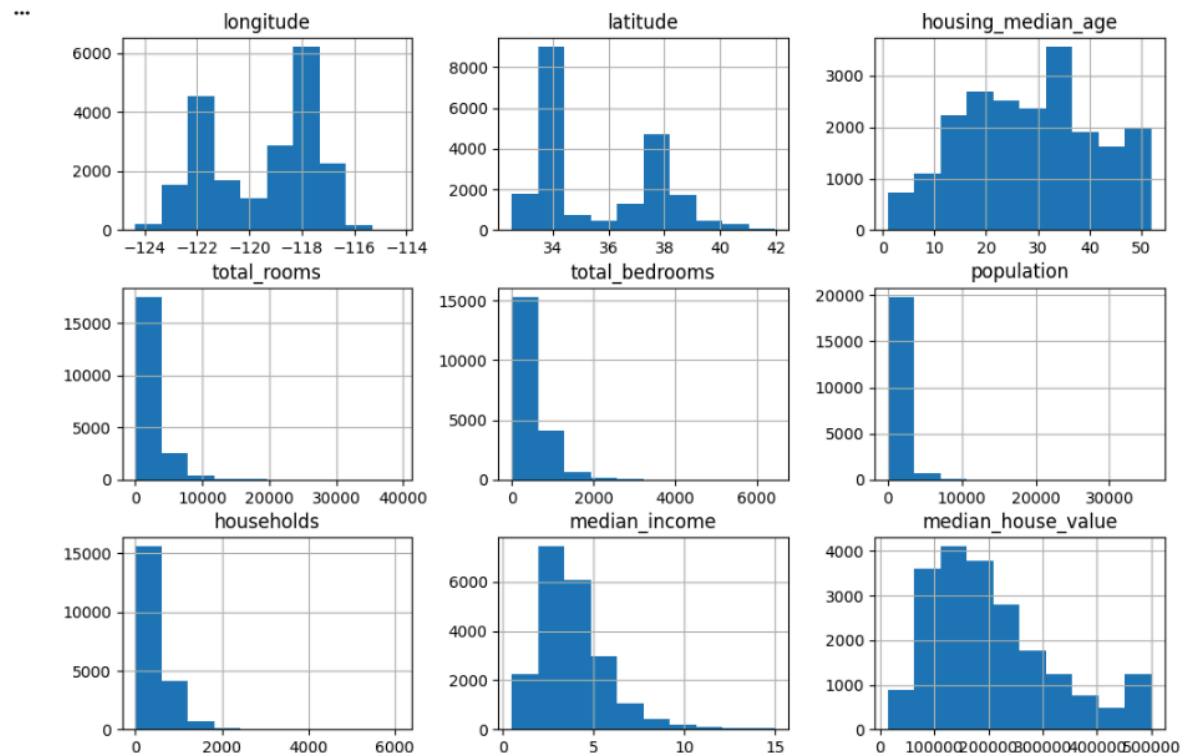
Se aplicó `.info()` y `.describe()` para identificar tipos de datos, conteos de valores no nulos y estadísticas descriptivas. Se detectó que `total_bedrooms` tiene 207 valores nulos (del total de 20,640 registros). Se graficaron histogramas de todas las variables para visualizar distribuciones y detectar sesgos y outliers. El hallazgo más relevante fue el límite artificial en `median_house_value` a 500,000 dólares: muchas viviendas caras quedaron registradas con el mismo precio máximo, creando una barra anormalmente alta en el histograma.

```
housing.info()
housing.describe()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   longitude              20640 non-null float64
1   latitude               20640 non-null float64
2   housing_median_age     20640 non-null float64
3   total_rooms            20640 non-null float64
4   total_bedrooms         20433 non-null float64
5   population             20640 non-null float64
6   households              20640 non-null float64
7   median_income          20640 non-null float64
8   median_house_value     20640 non-null float64
9   ocean_proximity        20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

|       | longitude    | latitude     | housing_median_age | total_rooms  | total_bedrooms | population   | households   | median_income | median_house_value |
|-------|--------------|--------------|--------------------|--------------|----------------|--------------|--------------|---------------|--------------------|
| count | 20640.000000 | 20640.000000 | 20640.000000       | 20640.000000 | 20433.000000   | 20640.000000 | 20640.000000 | 20640.000000  | 20640.000000       |
| mean  | -119.569704  | 35.631861    | 28.639486          | 2635.763081  | 537.870553     | 1425.476744  | 499.539680   | 3.870671      | 206855.816909      |
| std   | 2.003532     | 2.135952     | 12.585558          | 2181.615252  | 421.385070     | 1132.462122  | 382.329753   | 1.899822      | 115395.615874      |
| min   | -124.350000  | 32.540000    | 1.000000           | 2.000000     | 1.000000       | 3.000000     | 1.000000     | 0.499900      | 14999.000000       |
| 25%   | -121.800000  | 33.930000    | 18.000000          | 1447.750000  | 296.000000     | 787.000000   | 280.000000   | 2.563400      | 119600.000000      |
| 50%   | -118.490000  | 34.260000    | 29.000000          | 2127.000000  | 435.000000     | 1166.000000  | 409.000000   | 3.534800      | 179700.000000      |
| 75%   | -118.010000  | 37.710000    | 37.000000          | 3148.000000  | 647.000000     | 1725.000000  | 605.000000   | 4.743250      | 264725.000000      |
| max   | -114.310000  | 41.950000    | 52.000000          | 39320.000000 | 6445.000000    | 35682.000000 | 6082.000000  | 15.000100     | 500001.000000      |

```
housing.hist(figsize=(12,8))  
plt.show()
```



### 3. Tratamiento de Outliers y División Estratificada

Se tomaron dos decisiones críticas en esta etapa:

#### Filtrado de Outliers

Se eliminaron todos los registros donde `median_house_value`  $\geq$  500,000 dolares. Justificación: este tope artificial hace que viviendas con precios reales muy distintos (por ejemplo, 510,000 y 800,000 dolares) queden registradas con el mismo valor (500,000). Si se incluyen en el entrenamiento, confunden al modelo y aumentan el MAE. La eliminación reduce el ruido y mejora significativamente la calidad del aprendizaje.

## División Estratificada

Se uso `train_test_split` con `stratify` basado en categorias de ingreso medio (`income_cat`). Justificacion: el ingreso medio es el predictor mas correlacionado con el precio de la vivienda. Sin estratificacion, una division aleatoria podria producir conjuntos de entrenamiento y prueba con distribuciones de ingreso muy diferentes, generando sesgos en la evaluacion.

```
housing = housing[housing['median_house_value'] < 500000]
housing['income_cat'] = pd.cut(housing['median_income'],
                               bins=[0.,1.5,3.0,4.5,6.,np.inf], labels=[1,2,3,4,5])
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing['income_cat'], random_state=42)
```

## 4. Ingenieria de Caracteristicas

Se crearon tres nuevas variables de proporcion (ratios) sobre el conjunto de entrenamiento:

```
housing['rooms_per_household'] = housing['total_rooms'] / housing['households']
housing['bedrooms_ratio']      = housing['total_bedrooms'] /
housing['total_rooms']
housing['people_per_household'] = housing['population'] / housing['households']
```

Estas variables de proporcion son mas informativas que los valores absolutos porque normalizan las caracteristicas por el tamano del bloque censal, haciendo comparables bloques de diferente densidad poblacional. Por ejemplo, un bloque con 1,000 habitaciones y 500 hogares es muy diferente de uno con 1,000 habitaciones y 100 hogares.

## 5. Preparacion del Pipeline de Procesamiento

Se construyo un ColumnTransformer que aplica transformaciones distintas a variables numericas y categoricas:

```
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
full_pipeline = ColumnTransformer([
    ('num', num_pipeline, num_attribs),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_attribs)
])
```

La estrategia 'median' para imputacion es preferible a la media porque es robusta frente a outliers presentes en variables como total\_rooms y total\_bedrooms. El StandardScaler es necesario para modelos como Regresion Lineal que son sensibles a la escala de las variables.

## 6. Entrenamiento y Evaluacion con Validacion Cruzada

Se entrenaron cuatro modelos y se evaluo cada uno con 5-fold cross-validation usando MAE como metrica:

```
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42)
}
cv_scores = -cross_val_score(model, housing_prepared, housing_labels,
                              scoring='neg_mean_absolute_error', cv=5)
```

## 7. Optimización de Hiperparametros

Random Forest obtuvo el menor MAE CV. Se aplicó RandomizedSearchCV para encontrar la mejor combinación de hiperparametros:

```
param_dist = {  
    'n_estimators': [100, 200, 300],  
    'max_features': [2, 4, 6, 8, 10],  
    'max_depth': [None, 10, 20, 30]  
}  
  
rnd_search = RandomizedSearchCV(RandomForestRegressor(random_state=42),  
    param_distributions=param_dist, n_iter=10, cv=5,  
    scoring='neg_mean_absolute_error', random_state=42, n_jobs=-1)
```

## IV. ANALISIS E INTERPRETACION DE RESULTADOS

### Comparación de Modelos

Se La siguiente tabla resume los resultados de MAE en entrenamiento y validación cruzada para los cuatro modelos evaluados:

| Modelo            | MAE Entrenamiento<br>(\$) | MAE<br>Cross-Validation (\$) | Diferencia (\$) |
|-------------------|---------------------------|------------------------------|-----------------|
| Random Forest     | 11,182                    | 30,765                       | 19,583          |
| Gradient Boosting | 32,924                    | 34,207                       | 1,283           |
| Decision Tree     | 0                         | 42,877                       | 42,877          |
| Linear Regression | 43,635                    | 43,749                       | 114             |



## Interpretación por Modelo

- **Regresión Lineal:** Presenta MAE similar en entrenamiento (43,635) y CV (43,749), lo que indica que no tiene sobreajuste pero tampoco capacidad para capturar relaciones no lineales presentes en los datos. Es el peor modelo en términos de precisión absoluta.
- **Árbol de Decisión:** MAE de 0 en entrenamiento pero 42,877 en CV. Esto es un caso clásico de sobreajuste extremo: el modelo memoriza perfectamente los datos de entrenamiento pero no generaliza a datos nuevos.
- **Gradient Boosting:** MAE de 34,207 en CV con poca diferencia respecto al entrenamiento (32,924). Buen balance bias-varianza, pero menor precisión que Random Forest.
- **Random Forest:** Mejor MAE CV de 30,765. Aunque existe sobreajuste moderado (MAE entrenamiento de 11,182), el rendimiento en CV es el más alto. La promediación de múltiples árboles reduce la varianza y mejora la generalización.

- **Resultado de la Optimizacion de Hiperparametros**

| Hiperparametro | Valor Optimo | Justificacion                                              |
|----------------|--------------|------------------------------------------------------------|
| n_estimators   | 100          | Suficientes arboles para estabilizar el ensamble           |
| max_features   | 8            | Mayor numero de caracteristicas por split mejora precision |
| max_depth      | 30           | Arboles mas profundos capturan relaciones complejas        |

MAE CV tras optimizacion: 30,369 (mejora de ~400 dolares respecto al modelo base).

MAE Final en conjunto de prueba: 29,655 dolares.

El MAE final de 29,655 significa que el modelo comete un error promedio de aproximadamente 29,655 dolares al predecir el precio mediano de una vivienda. Considerando que los precios en el dataset oscilan entre 15,000 y 500,000 dolares, esto representa un error relativo de aproximadamente 10-15% del valor mediano de las viviendas, lo cual es un resultado competitivo para un modelo de Random Forest sin tecnicas avanzadas de boosting.



## V. CUESTIONARIO

### 1. ¿Por qué se usa validación cruzada en lugar de una simple división train/test?

La validación cruzada reduce la varianza de la estimación del error al promediar los resultados de evaluaciones independientes. Una simple división train/test depende fuertemente de que registros caen en cada conjunto, pudiendo dar estimaciones optimistas o pesimistas según la suerte de la partición. Con 5-fold CV, todos los datos sirven como validación exactamente una vez.

### 2. ¿Por qué Random Forest supera a Decision Tree en validación cruzada?

Un solo Árbol de Decisión se ajusta porque memoriza el ruido de los datos de entrenamiento (MAE entrenamiento = 0). Random Forest construye múltiples árboles sobre subconjuntos aleatorios de datos y características, promediando sus predicciones. Este proceso reduce la varianza del modelo (aunque aumenta ligeramente el sesgo), resultando en mejor generalización.





**3. Por que se filtran los registros con median\_house\_value >= 500,000?**

El dataset tiene un tope artificial a 500,000 dólares: viviendas que realmente valen entre 500,000 y 900,000 dólares quedaron registradas con el mismo valor de 500,000. Esto crea un ruido artificial que confunde al modelo. Al eliminar estos registros, el modelo aprende relaciones más limpias y consistentes entre las características y el precio real.

**4. ¿Cómo se aplica el pipeline al conjunto de prueba?**

Se usa `full_pipeline.transform(X_test)` (NO `fit_transform`). El pipeline ya fue ajustado (fit) durante el entrenamiento, por lo que los parametros de imputacion (mediana calculada en train) y escalado (media y desviacion estandar de train) se aplican directamente al conjunto de prueba. Usar `fit_transform` en test causaria data leakage.



## CONCLUSIONES

- El modelo Random Forest con optimización de hiperparámetros alcanzó el menor MAE en validación cruzada (30,369 dólares) y un MAE final de 29,655 dólares en el conjunto de prueba, siendo el mejor modelo para predecir precios de viviendas en California con las técnicas implementadas.
- El filtrado de outliers (valores topados a 500,000 dólares) fue la decisión de preprocesamiento con mayor impacto positivo en el MAE, al eliminar ruido artificial que distorsionaba el aprendizaje del modelo.
- La ingeniería de características (ratios rooms\_per\_household, bedrooms\_ratio, people\_per\_household) mejoró la capacidad predictiva al convertir valores absolutos en proporciones comparables entre bloques censales de diferente tamaño.
- El Árbol de Decisión demostró un sobreajuste extremo (MAE train = 0, MAE CV = 42,877), confirmando la importancia de usar validación cruzada para detectar modelos que memorizan en lugar de generalizar.
- El uso de Pipeline y ColumnTransformer garantiza la reproducibilidad y previene la fuga de datos (data leakage) al asegurar que las transformaciones se calculan solo sobre datos de entrenamiento.



## RECOMENDACIONES

- Explorar modelos de boosting mas avanzados como XGBoost, LightGBM o CatBoost, que tipicamente superan a Random Forest en datasets tabulares estructurados.
- Ampliar la busqueda de hiperparametros con  $n\_iter \geq 50$  o usar Optuna para una optimizacion bayesiana mas eficiente del espacio de hiperparametros.
- Incorporar variables geograficas adicionales como la distancia a centros urbanos principales (Los Angeles, San Francisco) usando las columnas longitude y latitude.
- Aplicar transformacion logaritmica a la variable objetivo (median\_house\_value) para reducir la asimetria de su distribucion y potencialmente mejorar el MAE.
- Considerar tecnicas de seleccion de caracteristicas (SelectKBest, importancia de caracteristicas de Random Forest) para identificar y eliminar variables redundantes o de bajo poder predictivo.



## BIBLIOGRAFÍA

- Geron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An Introduction to Statistical Learning (2nd ed.). Springer.
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

## WEBGRAFIA

- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- Dataset California Housing (GitHub - ageron): <https://github.com/ageron/data>
- Google Colaboratory: <https://colab.research.google.com>
- Pandas Documentation: <https://pandas.pydata.org/docs/>
- Matplotlib Documentation: <https://matplotlib.org/stable/>